



Department of Computer Science  
Faculty of Computing  
UNIVERSITI TEKNOLOGI MALAYSIA

|                                         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |                  |                  |                    |                   |
|-----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|------------------|--------------------|-------------------|
| <b>SUBJECT NAME:</b>                    | <b>COMPUTER ORGANIZATION AND ARCHITECTURE</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |                  |                  |                    |                   |
| <b>SUBJECT CODE:</b>                    | <b>SECR 1033</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |                  |                  |                    |                   |
| <b>SEMESTER:</b>                        | <b>2 – 2023/2024</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |                  |                  |                    |                   |
| <b>LAB TITLE:</b>                       | <b>Lab 2: Arithmetic Equations &amp; Operations</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |                  |                  |                    |                   |
| <b>STUDENT INFO :</b>                   | <p>Execute the lab in group of two.</p> <table><tr><td><b>Student 1</b></td><td><b>Student 2</b></td></tr><tr><td><i>No. 1, 3, 5</i></td><td><i>No 2, 4, 6</i></td></tr></table> <p><b>Name 1:</b> NG YU HIN</p> <p><b>Metric No:</b> A23CS0148</p> <p><b>Link for Video Demo:</b> <a href="https://youtu.be/wkEExyOsOZQ">https://youtu.be/wkEExyOsOZQ</a></p> <p><b>Name 2:</b> ELIJAH SHE YU SHENG</p> <p><b>Metric No:</b> A23CS0073</p> <p><b>Link for Video Demo:</b> <a href="https://youtu.be/wkEExyOsOZQ">https://youtu.be/wkEExyOsOZQ</a></p> | <b>Student 1</b> | <b>Student 2</b> | <i>No. 1, 3, 5</i> | <i>No 2, 4, 6</i> |
| <b>Student 1</b>                        | <b>Student 2</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |                  |                  |                    |                   |
| <i>No. 1, 3, 5</i>                      | <i>No 2, 4, 6</i>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |                  |                  |                    |                   |
| <b>SUBMISSION DATE<br/>&amp; ITEMS:</b> | <p><b>Duration of submission :-</b></p> <p>2 weeks</p> <p><b>Submission items in e-learning:</b></p> <p>1. Lab 2 exercise sheet/file (in .pdf), with the links for demo video 5 - 10min, on the cover page.</p> <p>2. The assembly programs (in .asm).</p>                                                                                                                                                                                                                                                                                             |                  |                  |                    |                   |

**MARKS:**

\_\_\_\_\_

## Arithmetic Equation Coding in Assembly Language

Q1. Execute the program below. Determine output of the program by inspecting the content of the related registers.

- a) Fill in Table 1 with the content of each register or variable on every LINE, in **Hexadecimal** (as per the output). Please complete the comments for every LINE.
- b) Paste the screenshot of all registers' content after each LINE is executed.

```
INCLUDE Irvine32.inc
.data
var1 word 1
var2 word 9

.code
main PROC
    mov ax, var1      ; LINE1
    mov bx, var2      ; LINE2
    xchg ax, bx       ; LINE3
    mov var1, ax      ; LINE4
    mov var2, bx      ; LINE5
    call DumpRegs
    exit
main ENDP
END main
```

### Answer Q1

- a) Fill (Write) in the contents for the related register in each line:

**Table 1**

|       |                                          |                                                                   |
|-------|------------------------------------------|-------------------------------------------------------------------|
| LINE1 | <b>AX = 0001h</b><br><b>var1 = 0001h</b> | Move the value of var1 (1d) into register AX                      |
| LINE2 | <b>BX = 0009h</b><br><b>var2 = 0009h</b> | Move the value of var2 (9d) into register BX                      |
| LINE3 | <b>AX = 0009h</b><br><b>BX = 0001h</b>   | Exchange the values between register AX (1d) and register BX (9d) |
| LINE4 | <b>AX = 0009h</b><br><b>var1 = 0009h</b> | Move the value of register AX (9d) into variable var1             |
| LINE5 | <b>BX = 0001h</b><br><b>var2 = 0001h</b> | Move the value of register BX (1d) into variable var2             |

- b) Paste here screenshot of all registers' content after each LINE is executed:

LINE1:

```
EAX = 00D60001 EBX = 00BDE000 ECX = 002B1104 EDX = 002B1104
ESI = 002B1104 EDI = 002B1104 EIP = 002B37B6 ESP = 00D6FA54
EBP = 00D6FA60 EFL = 00000246
```

LINE2:

```
EAX = 00D60001 EBX = 00BD0009 ECX = 002B1104 EDX = 002B1104
ESI = 002B1104 EDI = 002B1104 EIP = 002B37BD ESP = 00D6FA54
EBP = 00D6FA60 EFL = 00000246
```

LINE3:

```
EAX = 00D60009 EBX = 00BD0001 ECX = 002B1104 EDX = 002B1104
ESI = 002B1104 EDI = 002B1104 EIP = 002B37BF ESP = 00D6FA54
EBP = 00D6FA60 EFL = 00000246
```

LINE4:

```
EAX = 00D60009 EBX = 00BD0001 ECX = 002B1104 EDX = 002B1104
ESI = 002B1104 EDI = 002B1104 EIP = 002B37C5 ESP = 00D6FA54
EBP = 00D6FA60 EFL = 00000246
```

LINE5:

```
EAX = 00D60009 EBX = 00BD0001 ECX = 002B1104 EDX = 002B1104
ESI = 002B1104 EDI = 002B1104 EIP = 002B37CC ESP = 00D6FA54
EBP = 00D6FA60 EFL = 00000246
```

Q2. Execute the program below. Determine output of the program by inspecting the content of the related registers and watches.

- Fill in Table 2 with the content of each register or variable on every LINE, in **Hexadecimal** (as per the output). Please complete the comments for every LINE.
- Paste the screenshot of all registers' content after each LINE is executed.

**Arithmetic expression:  $Rval = (-Xval + (Yval - Zval)) + 1$**

```
include irvine32.inc

.data
Rval DWORD ?
Xval DWORD 26
Yval DWORD 30
Zval DWORD 40

.code
main proc
    mov eax,Xval        ; LINE1
    neg eax             ; LINE2
    mov ebx,Yval        ; LINE3
    sub ebx,Zval        ; LINE4
    add eax,ebx         ; LINE5
    inc eax             ; LINE6
    mov Rval,eax        ; LINE7
    exit
main endp
end main
```

### Answer Q2

- Fill (Write) in the contents for the related register in each line:

**Table 2**

|       |                                                     |                                                             |
|-------|-----------------------------------------------------|-------------------------------------------------------------|
| LINE1 | <b>EAX = 0000001Ah</b><br><b>Xval = 0000001Ah</b>   | Move the value of Xval (26d) into register EAX              |
| LINE2 | <b>EAX = FFFFFFFE6h</b>                             | Negate the value of register EAX in 2's complement          |
| LINE3 | <b>EBX = 0000001Eh</b><br><b>Yval = 0000001Eh</b>   | Move the value of Yval (30d) into register EBX              |
| LINE4 | <b>EBX = FFFFFFFF6h</b><br><b>Zval = 00000028h</b>  | Subtract the value of register EBX with value of Zval (40d) |
| LINE5 | <b>EAX = FFFFFFFDCh</b><br><b>EBX = FFFFFFFF6h</b>  | Add the value of register EAX with value of register EBX    |
| LINE6 | <b>EAX = FFFFFFFDDh</b>                             | Increment the value of register EAX by 1                    |
| LINE7 | <b>EAX = FFFFFFFDDh</b><br><b>Rval = FFFFFFFDDh</b> | Move the value stored in register EAX into Rval             |

b) Paste here screenshot of all registers' content after each LINE is executed:

LINE1: EAX = 0000001A EBX = 01054000 ECX = 00E1100A EDX = 00E1100A ESI = 00E1100A  
EDI = 00E1100A EIP = 00E11025 ESP = 0136F898 EBP = 0136F8A4 EFL = 00000246

LINE2: EAX = FFFFFFFE6 EBX = 01054000 ECX = 00E1100A EDX = 00E1100A ESI = 00E1100A  
EDI = 00E1100A EIP = 00E11027 ESP = 0136F898 EBP = 0136F8A4 EFL = 00000293  
0x00E14008 = 0000001E

LINE3: EAX = FFFFFFFE6 EBX = 0000001E ECX = 00E1100A EDX = 00E1100A ESI = 00E1100A  
EDI = 00E1100A EIP = 00E1102D ESP = 0136F898 EBP = 0136F8A4 EFL = 00000293  
0x00E1400C = 00000028

LINE4: EAX = FFFFFFFE6 EBX = FFFFFFFF6 ECX = 00E1100A EDX = 00E1100A ESI = 00E1100A  
EDI = 00E1100A EIP = 00E11033 ESP = 0136F898 EBP = 0136F8A4 EFL = 00000287

LINE5: EAX = FFFFFFFDC EBX = FFFFFFFF6 ECX = 00E1100A EDX = 00E1100A ESI = 00E1100A  
EDI = 00E1100A EIP = 00E11035 ESP = 0136F898 EBP = 0136F8A4 EFL = 00000283

LINE6: EAX = FFFFFFFDD EBX = FFFFFFFF6 ECX = 00E1100A EDX = 00E1100A ESI = 00E1100A  
EDI = 00E1100A EIP = 00E11036 ESP = 0136F898 EBP = 0136F8A4 EFL = 00000287  
0x00E14000 = 00000000

LINE7: EAX = FFFFFFFDD EBX = FFFFFFFF6 ECX = 00E1100A EDX = 00E1100A ESI = 00E1100A  
EDI = 00E1100A EIP = 00E1103B ESP = 0136F898 EBP = 0136F8A4 EFL = 00000287

Q3. Execute the program below. Determine output of the program by inspecting the content of the related registers.

- Fill in Table 3 with the content of each register or variable on every LINE, in **Hexadecimal** (as per the output). Please complete the comments for every LINE.
- Paste the screenshot of all registers' content after each LINE is executed.

**Arithmetic expression:  $\text{var4} = [(\text{var1} * \text{var2}) + \text{var3}] - 1$**

```
include irvine32.inc

.data
var1 DWORD 5
var2 DWORD 10
var3 DWORD 20
var4 DWORD ?

.code
main proc
    mov eax, var1        ; LINE1
    mul var2              ; LINE2
    add eax, var3         ; LINE3
    dec eax               ; LINE4
    exit
main endp
end main
```

### Answer Q3

- Fill (Write) in the contents for the related register in each line:

**Table 3**

|       |                                                   |                                                                        |
|-------|---------------------------------------------------|------------------------------------------------------------------------|
| LINE1 | <b>EAX = 00000005h</b><br><b>var1 = 00000005h</b> | Move the value of var1 (5d) into register EAX                          |
| LINE2 | <b>EAX = 00000032h</b><br><b>var2 = 0000000Ah</b> | Multiply the value of register EAX(5d) with the value of var2 (10d)    |
| LINE3 | <b>EAX = 00000046h</b><br><b>var3 = 00000014h</b> | Add the value of register EAX (50d) with the value of var3 (20d)       |
| LINE4 | <b>EAX = 00000045h</b><br><b>var4 = 00000045h</b> | Move the value of register EAX into variable var4 after decrement by 1 |

- Paste here screenshot of all registers' content after each LINE is executed:

LINE1: EAX = 00000005 EBX = 00C0A000 ECX = 00AC100A EDX = 00AC100A ESI = 00AC100A  
EDI = 00AC100A EIP = 00AC1025 ESP = 00BCF968 EBP = 00BCF974 EFL = 00000246  
0x00AC4004 = 0000000A

LINE2: EAX = 00000032 EBX = 00C0A000 ECX = 00AC100A EDX = 00000000 ESI = 00AC100A  
EDI = 00AC100A EIP = 00AC102B ESP = 00BCF968 EBP = 00BCF974 EFL = 00000202  
0x00AC4008 = 00000014

LINE3: EAX = 00000046 EBX = 00C0A000 ECX = 00AC100A EDX = 00000000 ESI = 00AC100A  
EDI = 00AC100A EIP = 00AC1031 ESP = 00BCF968 EBP = 00BCF974 EFL = 00000202

LINE4: EAX = 00000045 EBX = 00C0A000 ECX = 00AC100A EDX = 00000000 ESI = 00AC100A  
EDI = 00AC100A EIP = 00AC1032 ESP = 00BCF968 EBP = 00BCF974 EFL = 00000202  
0x00AC400C = 00000000

Q4. Execute the program below. Determine output of the program by inspecting the content of the related registers.

- Fill in Table 4 with the content of each register or variable on every LINE, in Hexadecimal (as per the output). Please complete the comments for every LINE.
- Paste the screenshot of all registers' content after each LINE is executed.

**Arithmetic expression:  $\text{var4} = (\text{var1} * 5) / (\text{var2} - 3)$**

```
include irvine32.inc
.data
    var1 WORD 40
    var2 WORD 10
    var4 WORD ?
.code
main proc
    mov ax,var1      ; LINE1
    mov bx,5         ; LINE2
    mul bx           ; LINE3
    mov bx,var2      ; LINE4
    sub bx,3         ; LINE5
    div bx           ; LINE6
    mov var4,ax      ; LINE7
    exit
main endp
end main
```

#### Answer Q4

- Fill (Write) in the contents for the related register in each line:

**Table 4**

|       |                                                             |                                                                                                                                                    |
|-------|-------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| LINE1 | <b>AX = 0028h</b><br><b>var1 = 0028h</b>                    | Move the value of var1 (40d) into register AX                                                                                                      |
| LINE2 | <b>BX = 0005h</b>                                           | Move the immediate value of 5d into register BX                                                                                                    |
| LINE3 | <b>AX = 00C8h</b><br><b>BX = 0005h</b>                      | Multiply the value of register AX with the value of register BX store the value in register DX                                                     |
| LINE4 | <b>BX = 000Ah</b><br><b>var2 = 000Ah</b>                    | Move the value of var2 (10d) into register BX                                                                                                      |
| LINE5 | <b>BX = 0007h</b>                                           | Subtract the value of register BX with the immediate value of 3d                                                                                   |
| LINE6 | <b>AX = 001Ch</b><br><b>BX = 0007h</b><br><b>DX = 0004h</b> | Divide the value of register AX with the value of register BX, the quotient is then stored in register AX, and the remainder stored in register DX |
| LINE7 | <b>AX = 001Ch</b><br><b>var4 = 001Ch</b>                    | Move the value of register AX into variable var4                                                                                                   |

b) Paste here screenshot of all registers' content after each LINE is executed:

LINE1: EAX = 00530028 EBX = 00288000 ECX = 00E5100A EDX = 00E5100A ESI = 00E5100A  
EDI = 00E5100A EIP = 00E51026 ESP = 0053FC18 EBP = 0053FC24 EFL = 00000246

LINE2: EAX = 00530028 EBX = 00280005 ECX = 00E5100A EDX = 00E5100A ESI = 00E5100A  
EDI = 00E5100A EIP = 00E5102A ESP = 0053FC18 EBP = 0053FC24 EFL = 00000246

LINE3: EAX = 005300C8 EBX = 00280005 ECX = 00E5100A EDX = 00E50000 ESI = 00E5100A  
EDI = 00E5100A EIP = 00E5102D ESP = 0053FC18 EBP = 0053FC24 EFL = 00000202

LINE4: EAX = 005300C8 EBX = 0028000A ECX = 00E5100A EDX = 00E50000 ESI = 00E5100A  
EDI = 00E5100A EIP = 00E51034 ESP = 0053FC18 EBP = 0053FC24 EFL = 00000202

LINE5: EAX = 005300C8 EBX = 00280007 ECX = 00E5100A EDX = 00E50000 ESI = 00E5100A  
EDI = 00E5100A EIP = 00E51038 ESP = 0053FC18 EBP = 0053FC24 EFL = 00000202

LINE6: EAX = 0053001C EBX = 00280007 ECX = 00E5100A EDX = 00E50004 ESI = 00E5100A  
EDI = 00E5100A EIP = 00E5103B ESP = 0053FC18 EBP = 0053FC24 EFL = 00000202

LINE7: EAX = 0053001C EBX = 00280007 ECX = 00E5100A EDX = 00E50004 ESI = 00E5100A  
EDI = 00E5100A EIP = 00E51041 ESP = 0053FC18 EBP = 0053FC24 EFL = 00000202

## **Short Notes for MUL CX and DIV BL:**

### **MUL CX**

- a. MUL always uses AX (or its extended versions EAX or RAX) as the implicit destination register.
- b. The operand size determines the size of the result:
  - i. Byte-sized operand: Result in AX
  - ii. Word-sized operand: Result in DX:AX
  - iii. Doubleword-sized operand (32-bit mode): Result in EDX:EAX
  - iv. Quadword-sized operand (64-bit mode): Result in RDX:RAX
- c. The upper half of the result (DX or EDX or RDX) holds any overflow bits.
- d. The Carry Flag (CF) is set if the upper half of the product is non-zero.

### **DIV BL**

- a. DIV always uses the DX:AX or EDX:EAX pair as the implicit dividend register.
  - b. The divisor is specified as the operand of the DIV instruction.
  - c. The quotient is stored in AX (for 16-bit division) or EAX (for 32-bit division).
  - d. The remainder is stored in DX.
  - e. Clear DX (or EDX for 32-bit division) before division to ensure a correct 16-bit or 32-bit dividend.
  - f. If the divisor is 0, a division error occurs.
  - g. The Overflow Flag (OF) is set if the quotient is too large to fit in the destination register.
- 

Q5. Given the following instructions as is Code Snippet 1.

- a) Write a full program to execute the Code Snippet 1.
- b) What are the contents of the related registers after Code Snippet 1 is executed? Paste the screenshot of DumpReg.

#### **; Code Snippet 1 (MUL CX)**

```
MOV DX, 0           ; Clear DX
MOV AX, 1000h        ; Load 1000h into AX
MOV CX, 25h          ; Load 25h into CX
MUL CX               ; Multiply AX by CX, storing the result in DX:AX
```



## Answer Q5

a) Screenshot of full program (.asm) :

```
TITLE LAB 2 Q5
; Author: NG YU HIN & ELIJAH SHE YU SHENG
; Date: 1 June 2024

include Irvine32.inc

.code
main proc

; Code Snippet 1 (MUL CX)

MOV DX, 0; Clear DX
MOV AX, 1000h; Load 1000h into AX
MOV CX, 25h; Load 25h into CX
MUL CX; Multiply AX by CX, storing the result in DX : AX

call DumpRegs;
exit

main ENDP

END main
```

b) Paste here the screenshot of the final registers' content (DumpReg):

```
Registers  7  X
EAX = 004F5000 EBX = 003FB000 ECX = 00780025 EDX = 00780002 ESI = 00781104
EDI = 00781104 EIP = 007837C4 ESP = 004FFDE4 EBP = 004FFDF0 EFL = 00000A07
```

Q6. Given the following instructions as is Code Snippet 2.

- Write a full program to execute the Code Snippet 2.
- What are the contents of the related registers after Code Snippet 2 is executed? Paste the screenshot of DumpReg.

**; Code Snippet 2 (DIV BL)**

```
MOV DX, 0      ; Clear DX to form the 16-bit dividend in DX:AX
MOV AX, 803h   ; Load the dividend (8003h) into AX
MOV BL, 10h    ; Load the divisor (10h) into BL
DIV BL         ; Divide DX:AX by BL, whereby AX=quotient & DX=remainder
```

**Answer Q6**

- Screenshot of full program (.asm) :

```
TITLE LAB 2 Q6
; Author:NG YU HIN & ELIJAH SHE YU SHENG
; Date: 1 June 2024

include Irvine32.inc

.code
main proc

; Code Snippet 2 (DIV BL)
MOV DX, 0; Clear DX to form the 16 - bit dividend in DX : AX
MOV AX, 803h; Load the dividend(803h) into AX
MOV BL, 10h; Load the divisor(10h) into BL
DIV BL; Divide DX : AX by BL, whereby AX = quotient & DX = remainder

call DumpRegs;
exit

main ENDP

END main
```

- Paste here the screenshot of the final registers' content (DumpReg):

```
Registers
EAX = 00390380 EBX = 005DA010
ECX = 00291104 EDX = 00290000
ESI = 00291104 EDI = 00291104
EIP = 002937C1 ESP = 0039F8DC
EBP = 0039F8E8 EFL = 00000246
```